

■■ Hoja de Referencia Rápida: Spring Data JPA & Hibernate

■ 1. Anotaciones Mapeo Entidad-Relacional (JPA)

Anotación	Propósito y Configuración	Ejemplo
<code>@Entity</code>	Marca la clase como Entidad relacional.	<code>@Entity</code>
<code>@Table</code>	Especifica el nombre físico de la tabla.	<code>@Table(name = "tbl_cliente")</code>
<code>@Id</code>	Define la llave primaria de la entidad.	<code>@Id</code>
<code>@GeneratedValue</code>	Estrategia de generación de ID.	<code>@GeneratedValue(strategy = ...)</code>
<code>@Column</code>	Mapea propiedad a columna SQL.	<code>@Column(name="email", unique=true)</code>
<code>@Transient</code>	Ignora la propiedad; no se persiste.	<code>@Transient private String tmp;</code>
<code>@Enumerated</code>	Mapea Enum (STRING o ORDINAL).	<code>@Enumerated(EnumType.STRING)</code>

<code>@CreatedDate</code>	Fecha automática de creación.	<code>@CreatedDate private LocalDateTime t;</code>
<code>@LastModifiedDate</code>	Fecha automática de edición.	<code>@LastModifiedDate private LocalDateTime t;</code>

■ 2. Hibernate Internals e Integración con Spring Data JPA

Hibernate es la implementación ORM estándar que ejecuta por debajo Spring Data JPA. Comprender sus mecanismos internos permite optimizar el rendimiento y prevenir cuellos de botella.

A. Ciclo de Vida de las Entidades en Hibernate

- **Transitorio (`Transient`):** Objeto creado con `new`, sin ID y sin vinculación al contexto de persistencia.
- **Persistente (`Managed`):** Entidad vinculada al contexto de persistencia (Caché L1). Hibernate rastrea todos sus cambios.
- **Desprendido (`Detached`):** Entidad con ID cuya transacción o sesión ya ha finalizado.
- **Eliminado (`Removed`):** Marcado para ser eliminado de la base de datos al hacer `flush()`.

B. Mecanismo de Dirty Checking (Detección Sucia)

Dentro de un método `@Transactional`, Hibernate detecta automáticamente los cambios en las entidades en estado `Managed` y emite el `UPDATE` correspondiente al cerrar la transacción sin necesidad de invocar `repository.save()`.

```
@Transactional
public void actualizarEmail(Long id, String nuevoEmail) {
    // Entidad entra en estado Managed
    Cliente cliente = clienteRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("No existe"));

    // Dirty Checking: Hibernate detecta el setter
```

```
// y genera UPDATE automaticamente al salir del metodo
cliente.setEmail(nuevoEmail);
}
```

C. Diagnóstico y Solución del Problema N+1 SELECT

Ocurre cuando se consulta una entidad con una relación @OneToMany LAZY, provocando 1 consulta inicial y \$N\$ consultas secundarias adicionales.

```
// Solucion 1: JOIN FETCH en JPQL
@Query("SELECT c FROM Cliente c JOIN FETCH c.facturas")
List<Cliente> obtenerClientesConFacturas();

// Solucion 2: @EntityGraph en el Repositorio
@EntityGraph(attributePaths = {"facturas"})
List<Cliente> findByActivoTrue();
```

D. Propiedades Clave de Hibernate en application.yml

```
spring:
  jpa:
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        use_sql_comments: true
    jdbc:
      batch_size: 25
```

■ 3. Mapeo de Relaciones entre Entidades

```
@Entity
@Table(name = "Cliente")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

// Relacion One-to-Many LAZY
@OneToMany(mappedBy = "cliente",
            cascade = CascadeType.ALL,
            orphanRemoval = true,
            fetch = FetchType.LAZY)
private List<Factura> facturas = new ArrayList<>();

public void agregarFactura(Factura factura) {
    facturas.add(factura);
    factura.setCliente(this);
}
}

```

```

@Entity
@Table(name = "Factura")
public class Factura {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "cliente_id", nullable = false)
    private Cliente cliente;
}

```

■ 4. Consultas en Repositorios (`JpaRepository`)

```

@Repository
public interface ProductoRepository
    extends JpaRepository<Producto, Long> {

    // Consulta Derivada
    List<Producto> findByCategoriaAndActivoTrue(String cat);

    // Consulta JPQL
    @Query("SELECT p FROM Producto p " +
           "WHERE p.categoria = :cat AND p.stock > :min")
    List<Producto> buscarDisponibles(

```

```

        @Param("cat") String cat,
        @Param("min") Integer min
    );
}

// Operacion de Modificacion Masiva
@Modifying
@Transactional
@Query("UPDATE Producto p SET p.activo = false " +
        "WHERE p.stock = 0")
int desactivarAgotados();

// Stored Procedure Nativo
@Procedure(procedureName = "sp_obtener_productos")
List<Producto> spObtenerPorCategoria(
    @Param("p_cat") String cat
);
}

```

■ 5. Paginación y Ordenamiento

```

@Service
public class ProductoService {

    private final ProductoRepository repository;

    public ProductoService(ProductoRepository repository) {
        this.repository = repository;
    }

    @Transactional(readonly = true)
    public Page<ProductoDTO> listarPaginado(int page,
                                            int size,
                                            String orden) {

        Pageable pageable =
            PageRequest.of(page, size,
                           Sort.by(Sort.Direction.ASC, orden));

        Page<Producto> pageResult =
            repository.findAll(pageable);

        return pageResult.map(p ->
            new ProductoDTO(p.getId(), p.getNombre(), p.getPrecio())
        );
    }
}

```

```
    );  
    }  
}
```

■ 6. Control de Transacciones (@Transactional)

```
@Service  
public class TransferenciaService {  
    @Transactional(rollbackFor = Exception.class)  
    public void procesarPago(Long origen,  
                             Long destino,  
                             BigDecimal monto) {  
        // Lógica de negocio ejecutada bajo transacción ACID  
    }  
  
    @Transactional(readOnly = true)  
    public ClienteDTO consultarCliente(Long id) {  
        return clienteRepository.findById(id)  
            .map(ClienteDTO::new)  
            .orElseThrow(() -> new RuntimeException("No existe"));  
    }  
}
```